УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
Трг Доситеја Обрадовића 6, 21000 Нови Сад, Република Србија
Деканат: 021 6350-413; 021 450-810; Централа: 021 485 2000
Рачуноводство: 021 458-220; Студентска служба: 021 6350-763
Телефакс: 021 458-133; e-mail: ftndean@uns.ac.rs
ИНТЕГРИСАНИ СИСТЕМ МЕНАЏМЕНТА СЕРТИФИКОВАН ОД:

Projektovanje složenih digitalnih sistema

Tema projekta:

Pronalaženje optimalne putanje na 2D grid mapi zasnovane na GBFS algoritmu

Autori:

Uroš Cvijić, EE11/2022

Marko Horstmann, EE27/2022

Mentor:

MSc Jana Janković

Sadržaj

1. Uvod *4*

1.1 GBFS algoritam i organizacija podataka u memoriji *4*

Definicija: *4*

Formalna specifikacija: *4*

1.2 Primer ulaza i izlaza algoritma *5*

1.3 Primer ulaza i izlaza IP bloka *5*

2. Arhitektura i paralelizacija algoritma *6*

2.1 Uklanjanje petlji *6*

2.2 ASM dijagram *8*

2.3 Datapath sistema *16*

2.4 Paralelizacija pretrage mape *17*

2.5 Blok dijagram sistema sa paralelizacijom pretrage *17*

3. Interfejsi sistema *18*

3.1 Generički parametri sistema *18*

3.2 Ulazno-izlazni interfejsi *18*

4. Test benč i provera osnovne funkcionalnosti IP-a *19*

4.1 Tok simulacije *19*

4.2 Provera rezultata simulacije *20*

5. Implementacija AXI interfejsa *22*

5.1 Registarski interfejs *22*

5.2 Generički parametri AXI omotača *22*

5.3 Verifikacija AXI verzije *22*

6. Pakovanje sistema u IP jezgra *23*

6.1 Glavno IP jezgro *23*

6.2 Pomoćna IP jezgra *23*

6.3 Verifikacija pomoćnih modula *23*

7. Blok dizajn sistema *24*

7.1 Topologija sistema *24*

7.2 Konfiguracija memorijskih blokova *25*

7.3 Prilagođenje širine podataka *25*

7.4 Adresni prostor *26*

7.5 Prekidni signal *26*
7.6 Validacija dizajna *26*
8. Analiza integrisanog sistema *27*
8.1 Analiza utrošenih resursa *27*
8.2 Analiza kritičnih putanja i maksimalne frekvencije *27*
8.3 Određivanje propusne moći *28*
8.4 Poređenje sa predviđanjima sa predmeta PEUNSN *28*
8.4.1 Poređenje na nivou cele mape *29*
9. Automatizacija izgradnje sistema *29*
10. Bare-metal aplikacija u Vitis alatu *30*
10.1 Urađeni deo *30*
10.2 Šta je preostalo *30*
10.3 Razlozi zbog kojih deo nije završen *30*

1. Uvod

Cilj ovog projekta je projektovanje složenog digitalnog sistema za ubrzavanje proračuna GBFS algoritma, zasnovanog na softverskoj virtuelnoj platformi, odnosno ESL modelu. IP blok koji se projektuje u ovom projektu obavlja proračune koji su u softverskoj specifikaciji obuhvaćeni u moveNode() i evaluateSurroundingNodes() funkcijama.

1.1 GBFS algoritam i organizacija podataka u memoriji

Definicija:
Greedy Best-First Search predstavlja heuristički algoritam pretrage koji u svakom koraku bira za ekspanziju onaj čvor koji, prema heurističkoj proceni, ima najmanju preostalu udaljenost do ciljnog čvora.

Formalna specifikacija:
Algoritam koristi funkciju evaluacije definisanu kao:

$$f(n) = h(n)$$

gde je:

- n – trenutni čvor,
- $h(n)$ – heuristička funkcija koja procenjuje trošak najkraćeg puta od čvora n do ciljnog čvora.

U ovoj implementaciji se koristi heuristika pogodna za mreže u kojima je kretanje dozvoljeno u osam smerova (horizontalno, vertikalno i dijagonalno). Formula za izračunavanje heuristike glasi:

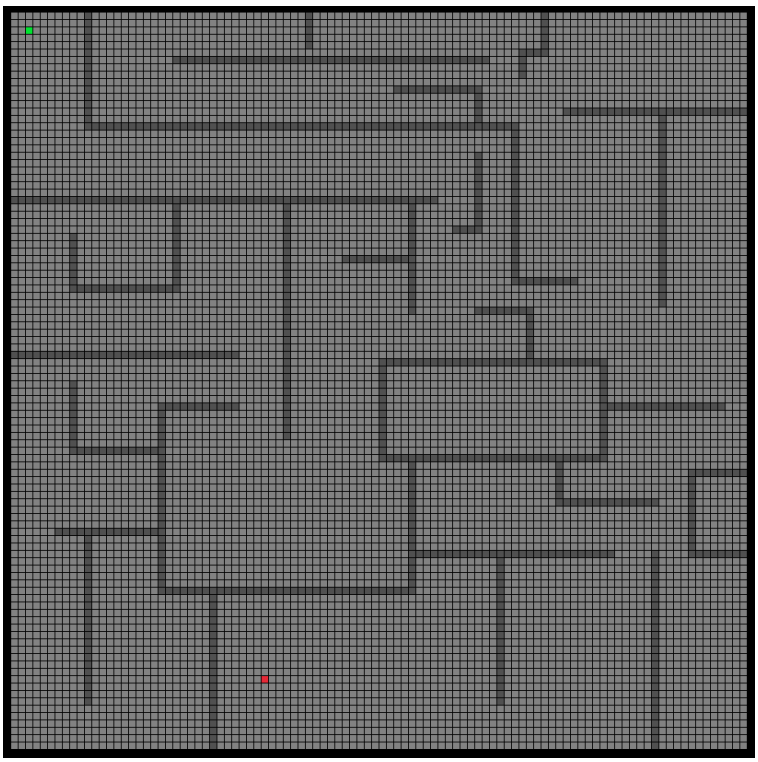
$$h(n) = 2 \times |dx - dy| + 3 \times \min(dx, dy)$$

gde su dx = |endX – x| i dy = |endY – y| apsolutne razlike koordinata trenutnog i ciljnog čvora.

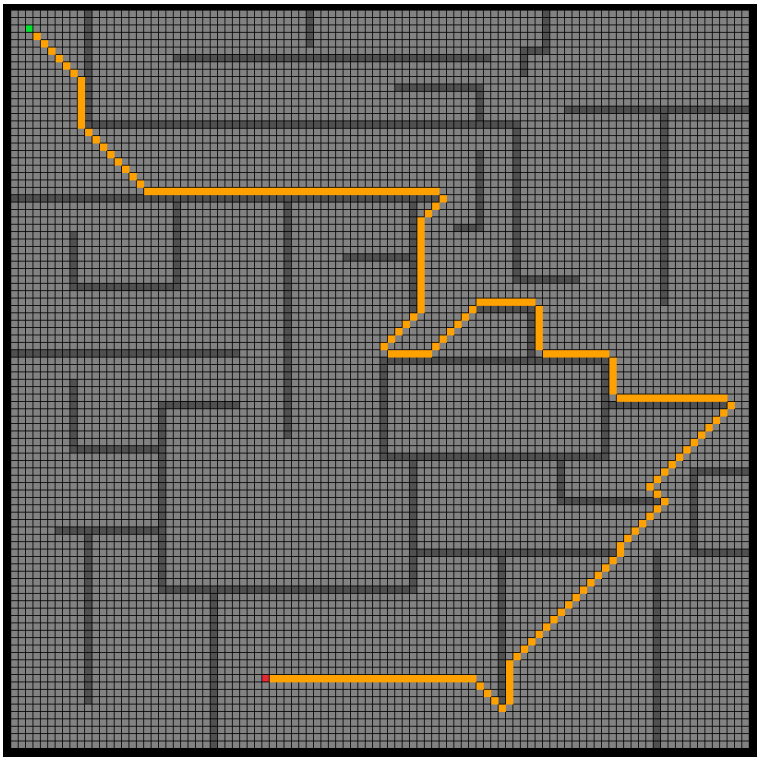
Podaci o svakom čvoru kodovani su u 24-bitnu strukturu unutar memorije:

- **distance [23:15]** (9 bita): Čuva izračunatu heurističku vrednost (1–297) ili specijalne sentinel vrednosti: 0 (neposećen čvor), 298 (zid/prepreka), 299 (početni čvor), 300 (ciljni čvor).
- **parentX [14:8]** (7 bita): X koordinata roditeljskog čvora (opseg 0–99).
- **parentY [7:1]** (7 bita): Y koordinata roditeljskog čvora (opseg 0–99).
- **visited [0]** (1 bit): Indikator da li je čvor već posećen.

1.2 Primer ulaza i izlaza algoritma

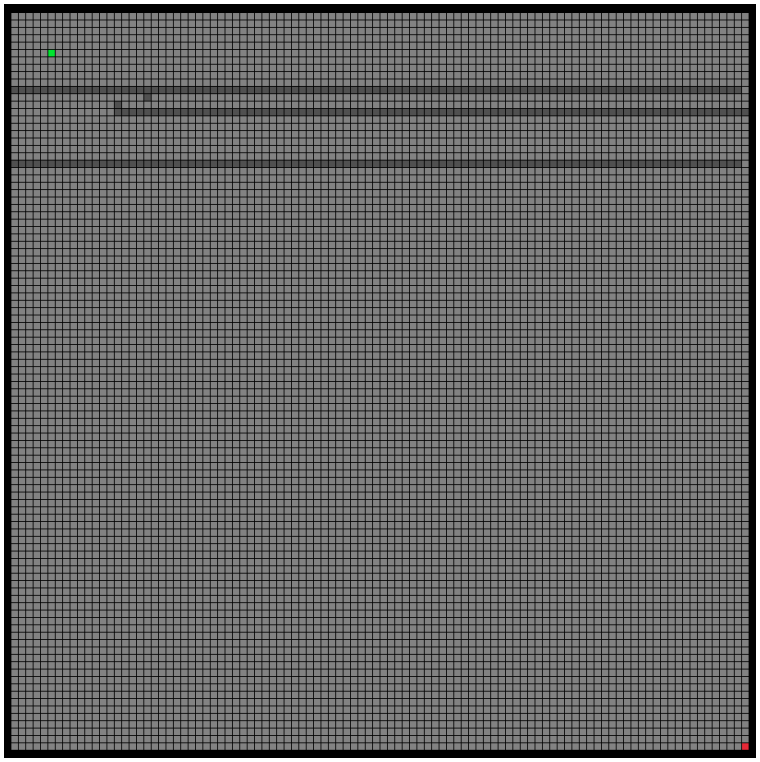


1. Primer ulaza sistema

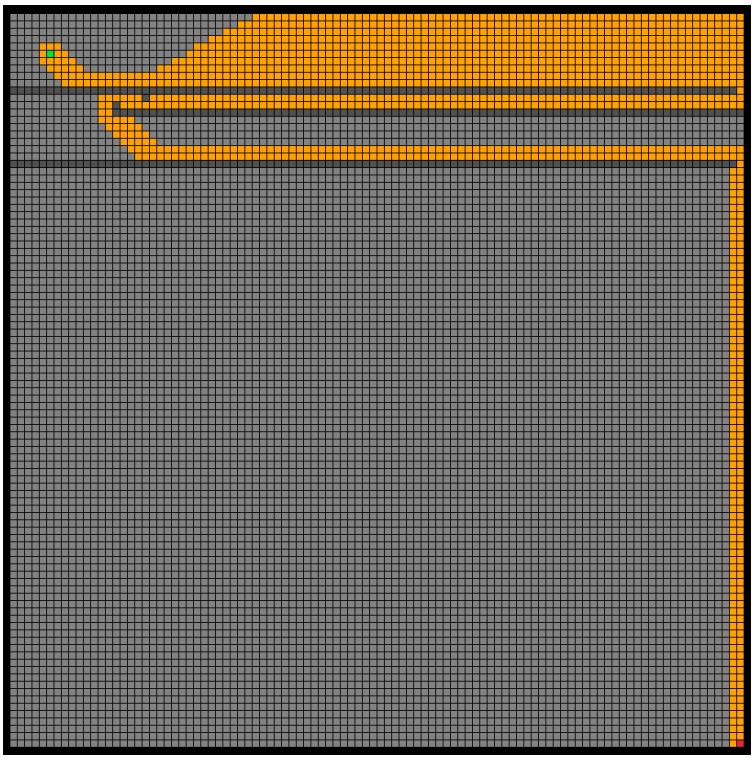


2. Primer izlaza sistema

1.3 Primer ulaza i izlaza IP bloka



1. Primer ulaza IP-a



2. Primer izlaza IP-a

Na prethodne 2 slike vidi se da je IP uspešno obradio mapu i došao do cilja. Bitno je naglasiti razliku između ovog prikaza i primera ulaza i izlaza čitavog algoritma (1.2). IP blok ne određuje samu konačnu putanju, već izvršava iterativni deo algoritma koji se zaustavlja kada se "dotakne" *end node* (Više u poglavlju 4.2). Procesor zatim iz memorije prati listu roditeljskih čvorova od kraja do početka, koja predstavlja traženu krajnju putanju.

2. Arhitektura i paralelizacija algoritma

Hardverski akcelerator realizovan je kao automat stanja (FSM). Za razliku od softverske implementacije gde se koordinate čvorova eksplicitno čuvaju u memoriji, hardverska arhitektura koristi matematičko preslikavanje adrese čvora:

$$\boldsymbol{Adresa = X \times 100 + Y}$$

Čime se štedi memorijski prostor u BRAM-u i eliminiše potreba za dodatnim registrima koordinata.

2.1 Uklanjanje petlji

Kako bi se algoritam preveo iz softvera u hardver potrebno je zameniti *for* i *while* petlje sa *if-goto* obrascima. Uklanjanje petlji iz *evaluateSurroundingNodes()* funkcije:

```
EVAL_INIT:
  i = currentX - 1;
EVAL_I_LOOP:
  if (i > currentX + 1) goto EVAL_DONE;
  j = currentY - 1;
EVAL_J_LOOP:
  if (j > currentY + 1) {
    i = i + 1;
    goto EVAL_I_LOOP;
  }
  if (i < 0 || i >= GRID_SIZE) { j = j + 1; goto EVAL_J_LOOP; }
  if (j < 0 || j >= GRID_SIZE) { j = j + 1; goto EVAL_J_LOOP; }
  if (i == currentX && j == currentY) { j = j + 1; goto EVAL_J_LOOP; }
  a = j * GRID_SIZE + i;
  d = distance[a];
  if (d == VAL_END) {
    parentX[a]    = currentX;
    parentY[a]    = currentY;
    parentValid[a] = 1;
    solved = 1;
    j = j + 1;
    goto EVAL_J_LOOP;
  }
  if (d == VAL_WALL || d == VAL_S TART) { j = j + 1; goto EVAL_J_LOOP; }
  dx = endX - i; if (dx < 0) dx = -dx;
  dy = endY - j; if (dy < 0) dy = -dy;
  if (dx > dy) h = 2 * (dx - dy) + 3 * dy;
  else       h = 2 * (dy - dx) + 3 * dx;
  distance[a] = h;
  if (!parentValid[a]) {
    parentX[a]    = currentX;
    parentY[a]    = currentY;
    parentValid[a] = 1;
  }
  j = j + 1;
  goto EVAL_J_LOOP;
EVAL_DONE:
```

Uklanjanje petlji iz *moveNode()* funkcije:

```
MOVE_INIT:
   minDist = 100000;
   nextX   = 0;
   nextY   = 0;
   found   = 0;
   x  = 0;
MOVE_X_LOOP:
   if (x >= GRID_SIZE) goto MOVE_CHECK;
   y = 0;
MOVE_Y_LOOP:
   if (y >= GRID_SIZE) {
      x = x + 1;
      goto MOVE_X_LOOP;
   }
   a = y * GRID_SIZE + x;
   if (!visited[a]) {
      dist = distance[a];
      if (dist > 0 && dist < VAL_WALL && dist < minDist) {
         minDist = dist;
         nextX   = x;
         nextY   = y;
         found   = 1;
      }
   }
   y = y + 1;
   goto MOVE_Y_LOOP;
MOVE_CHECK:
   if (found) {
      visited[currentY * GRID_SIZE + currentX] = 1;
      currentX = nextX;
      currentY = nextY;
   } else {
      solved = 1;
   }
   goto LOOP;
END:
```

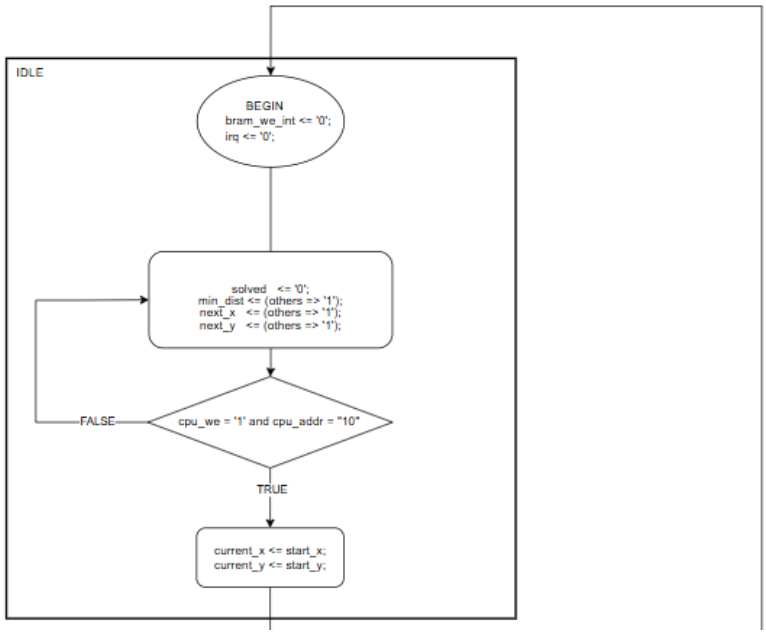
2.2 ASM dijagram

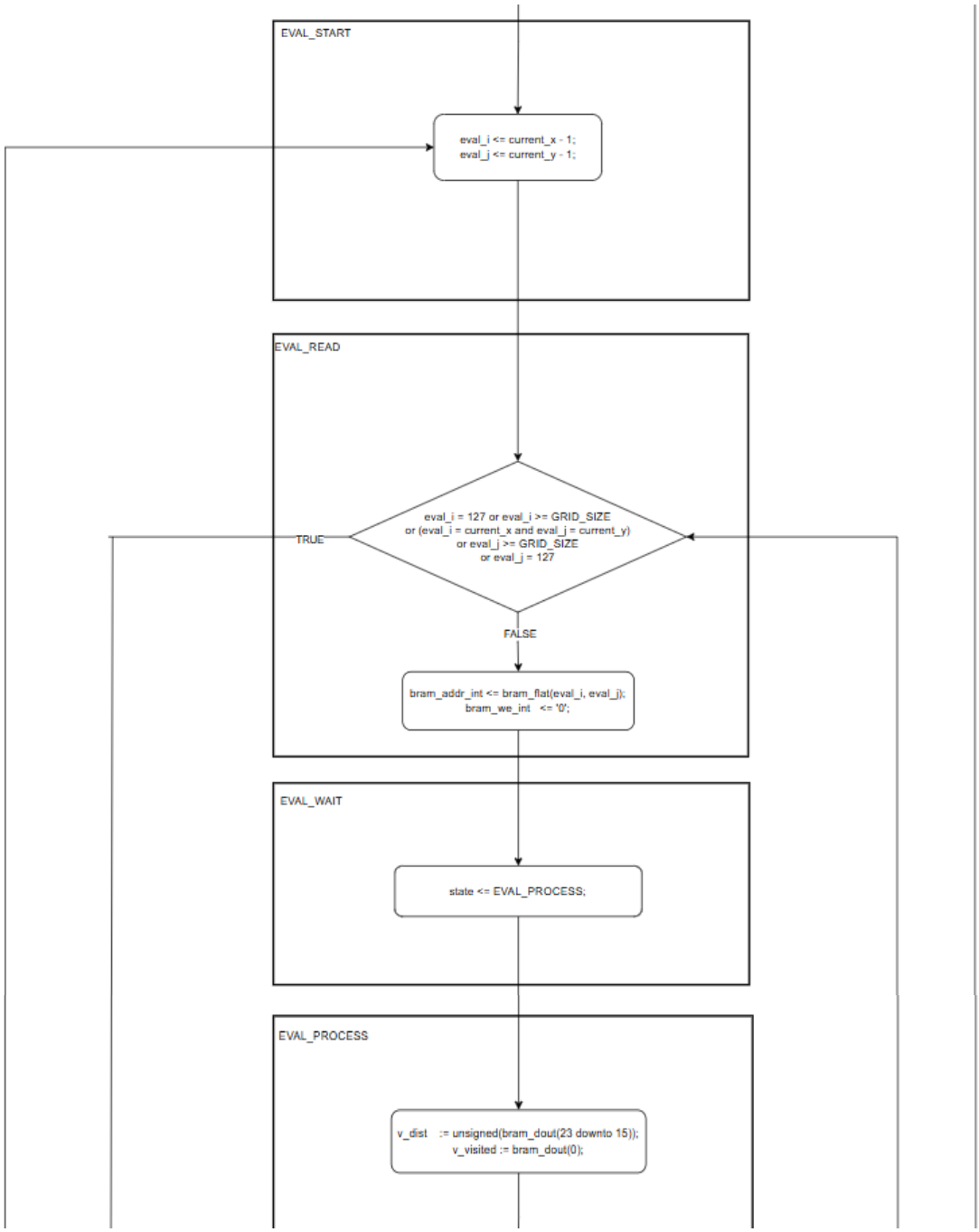
Algorithm state machine dijagram pokazuje stanja u kojima se sistem može naći tokom izvršavanja algoritma, kao i uslove pod kojima se prelazi iz jednog stanja u drugo.
Stanja se prirodno dele u dve grupe, koje odgovaraju dvema funkcijama softverske specifikacije pomenutim u uvodu. Stanja sa prefiksom EVAL realizuju posao funkcije *evaluateSurroundingNodes()*, odnosno obilaze osam suseda trenutnog čvora i računaju im heuristiku. Stanja sa prefiksom MOVE realizuju posao funkcije *moveNode()*, odnosno pretražuju mapu u potrazi za neposećenim čvorom sa najmanjom heuristikom, koji postaje sledeći trenutni čvor.

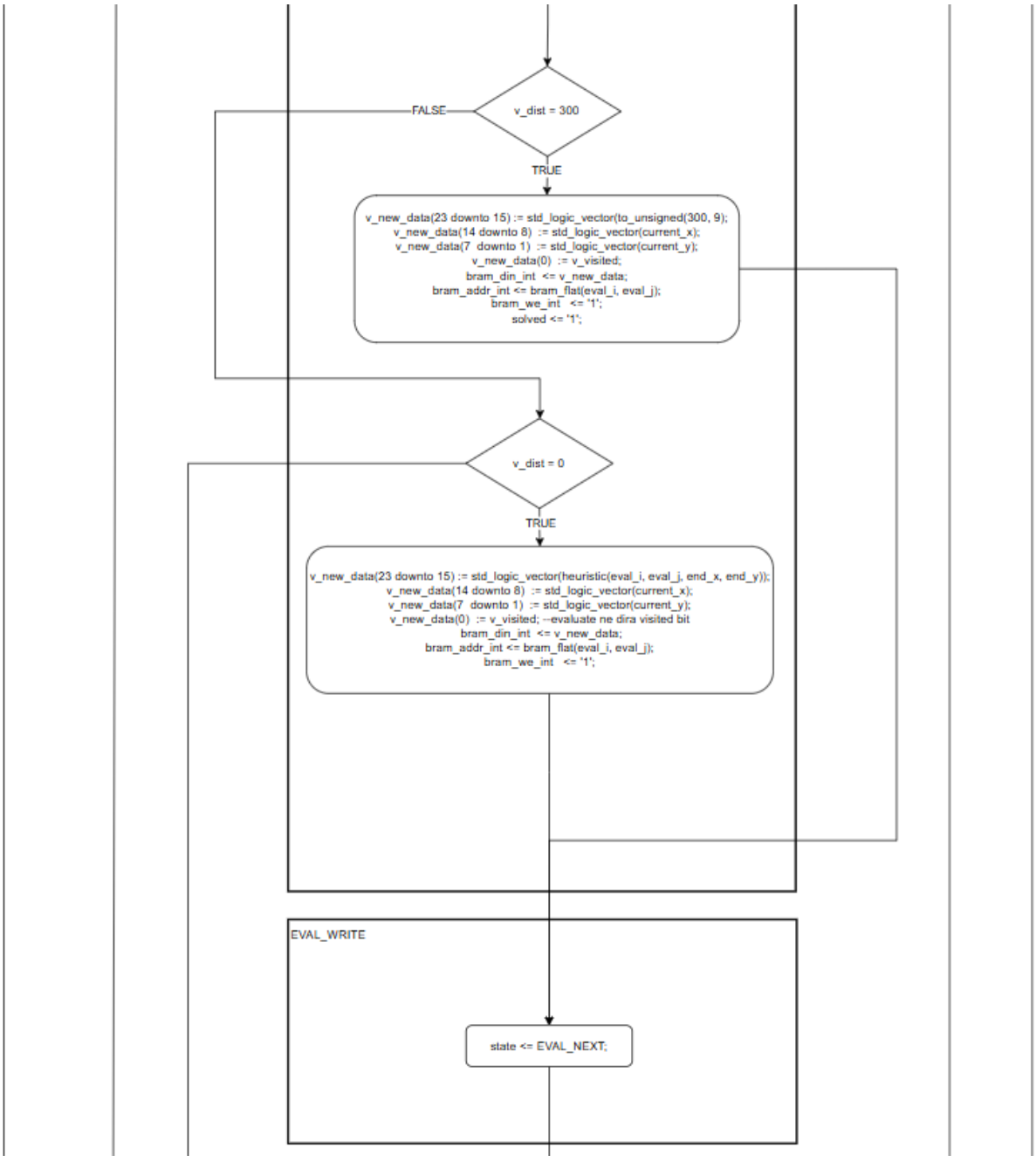
Rad automata je cikličan. Nakon što se u EVAL fazi obrade svi susedi, prelazi se u MOVE fazu, a po izboru sledećeg čvora automat se vraća na početak EVAL faze. Ovaj ciklus se ponavlja sve dok se ne dođe do ciljnog čvora ili dok se ne utvrdi da mapa nije rešiva, kada automat prelazi u stanje DONE.

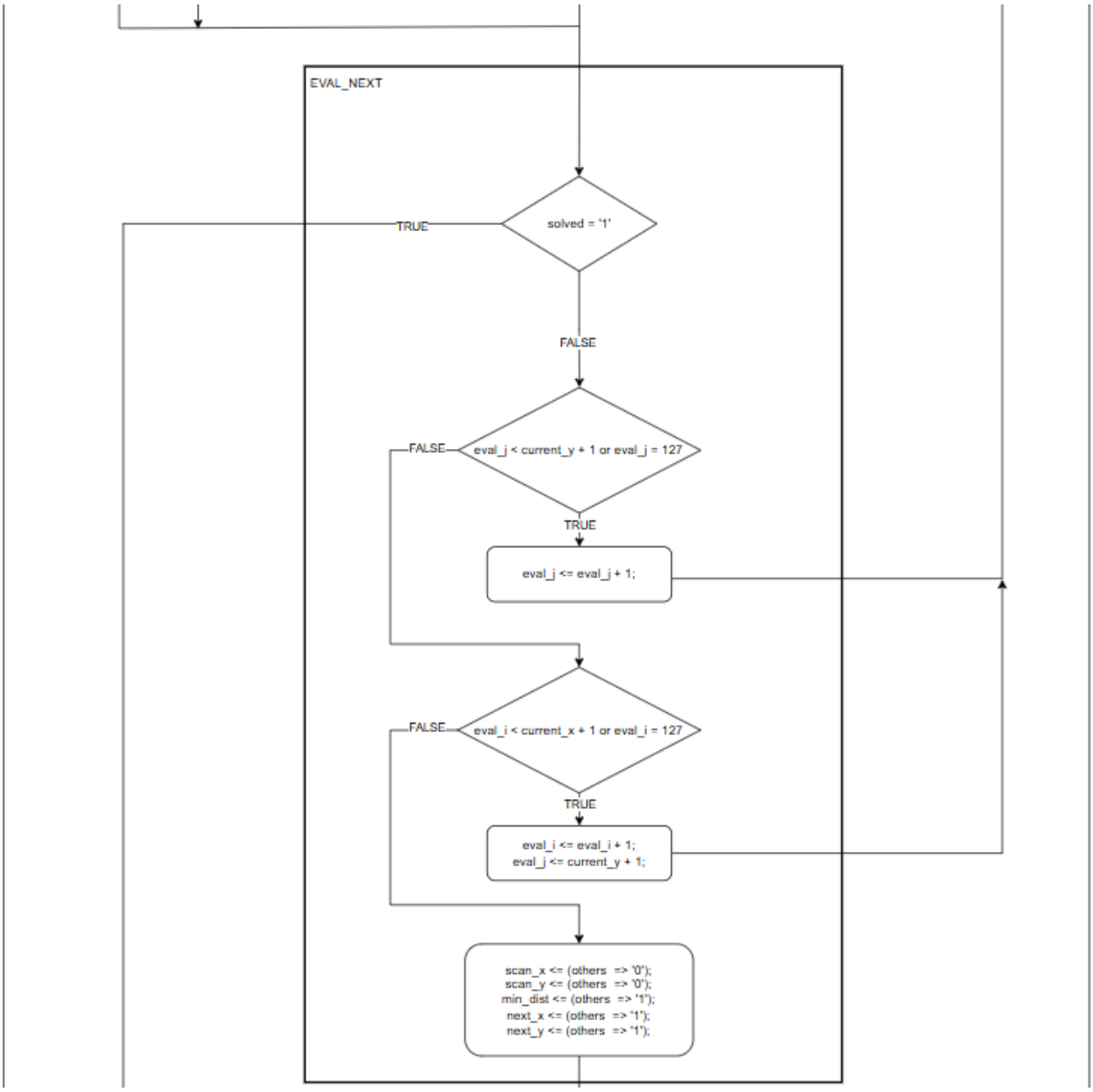
Poseban razlog za uvođenje pojedinih stanja je kašnjenje pri čitanju iz memorije. Pošto je čitanje iz BRAM-a sinhrono, podatak postaje validan tek jedan takt nakon postavljanja adrese. Zbog toga i EVAL i MOVE faza imaju svoje stanje čekanja, EVAL_WAIT i MOVE_WAIT, koja ne obavljaju nikakav proračun, već samo troše taj jedan takt. Iz istog razloga su upis i priprema podataka razdvojeni u zasebna stanja, umesto da se obave u jednom.

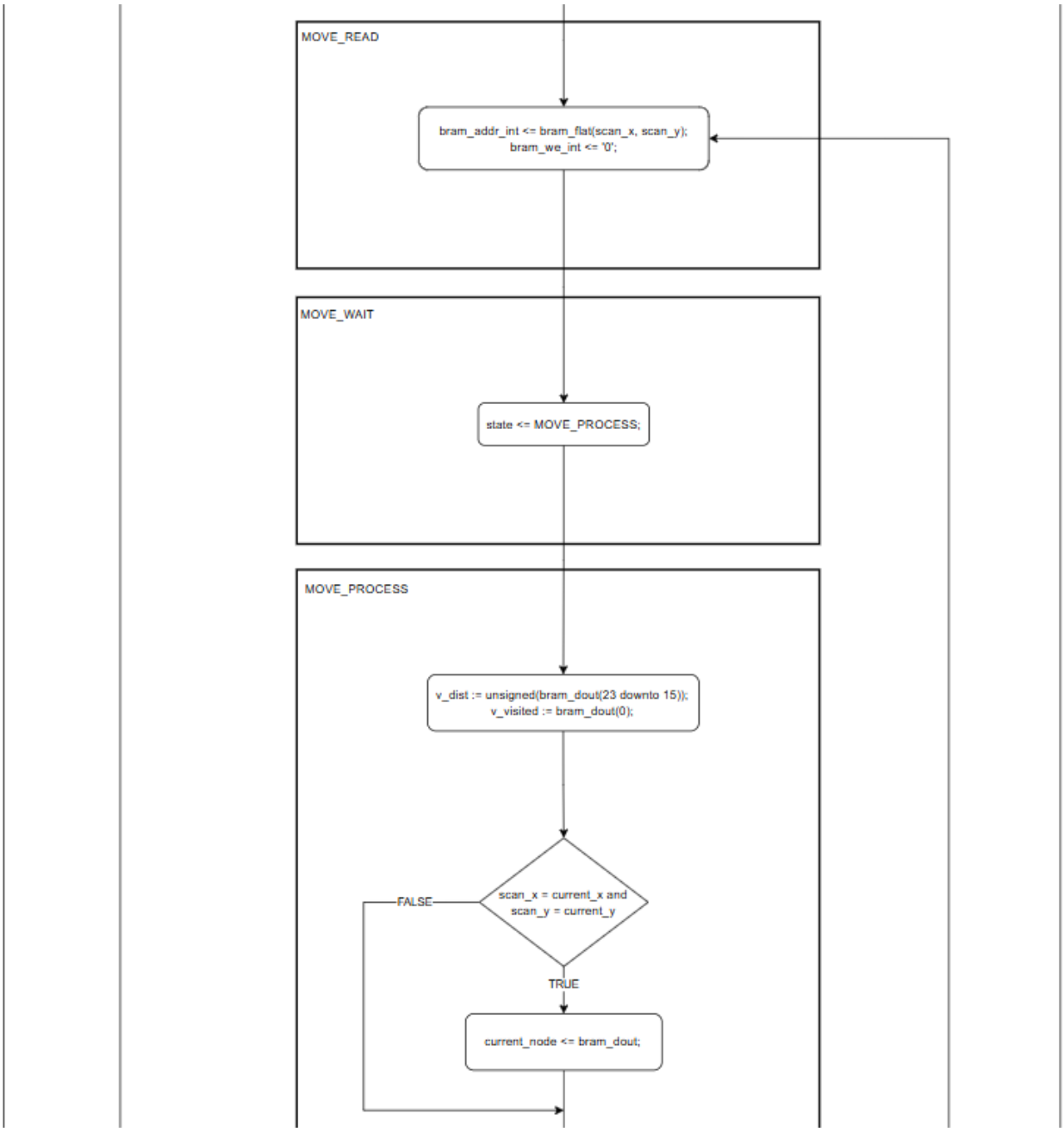
Napomena: Dijagram prikazuje osnovnu verziju automata. Izmene uvedene paralelizacijom su opisane u nastavku.

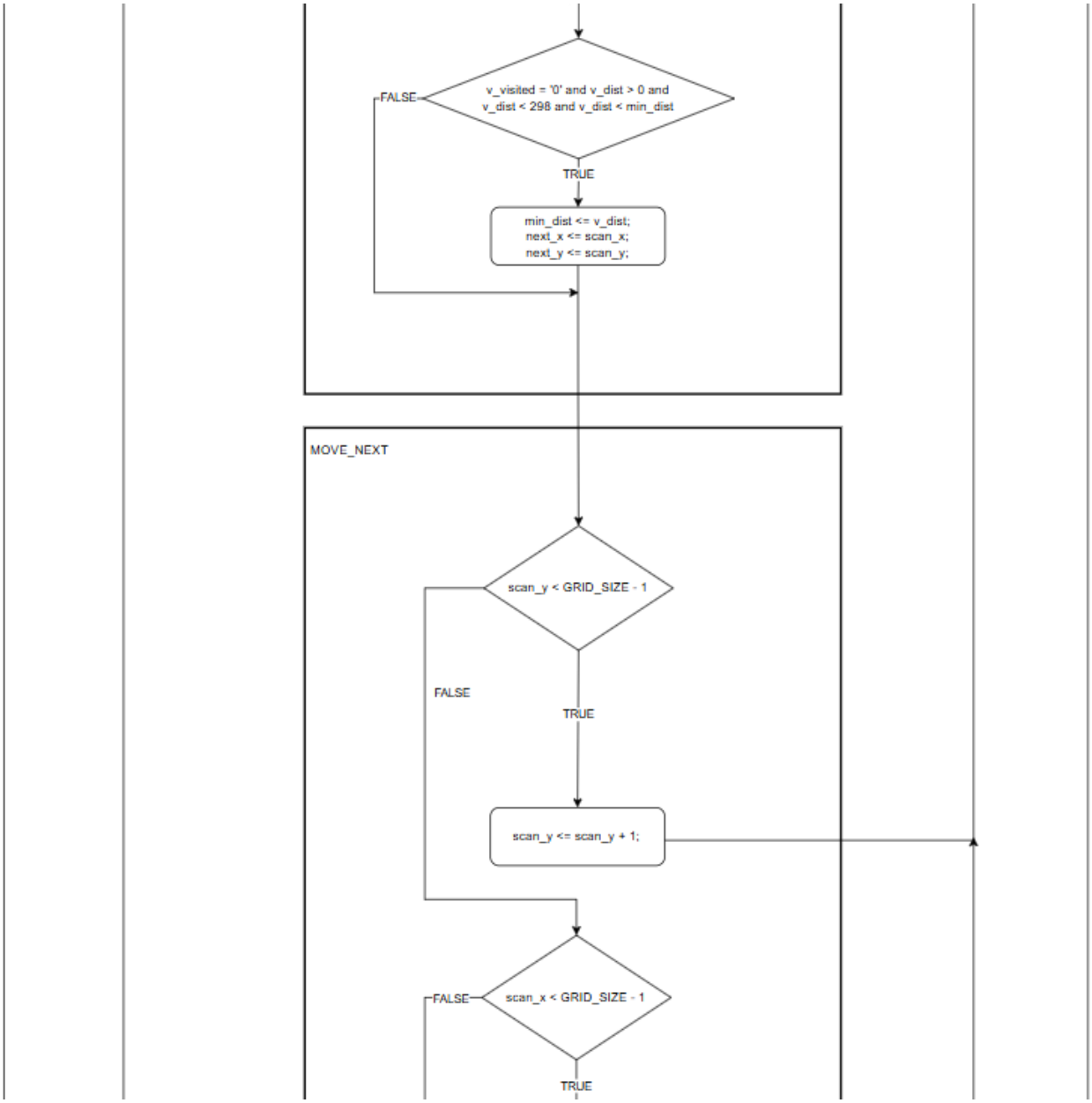


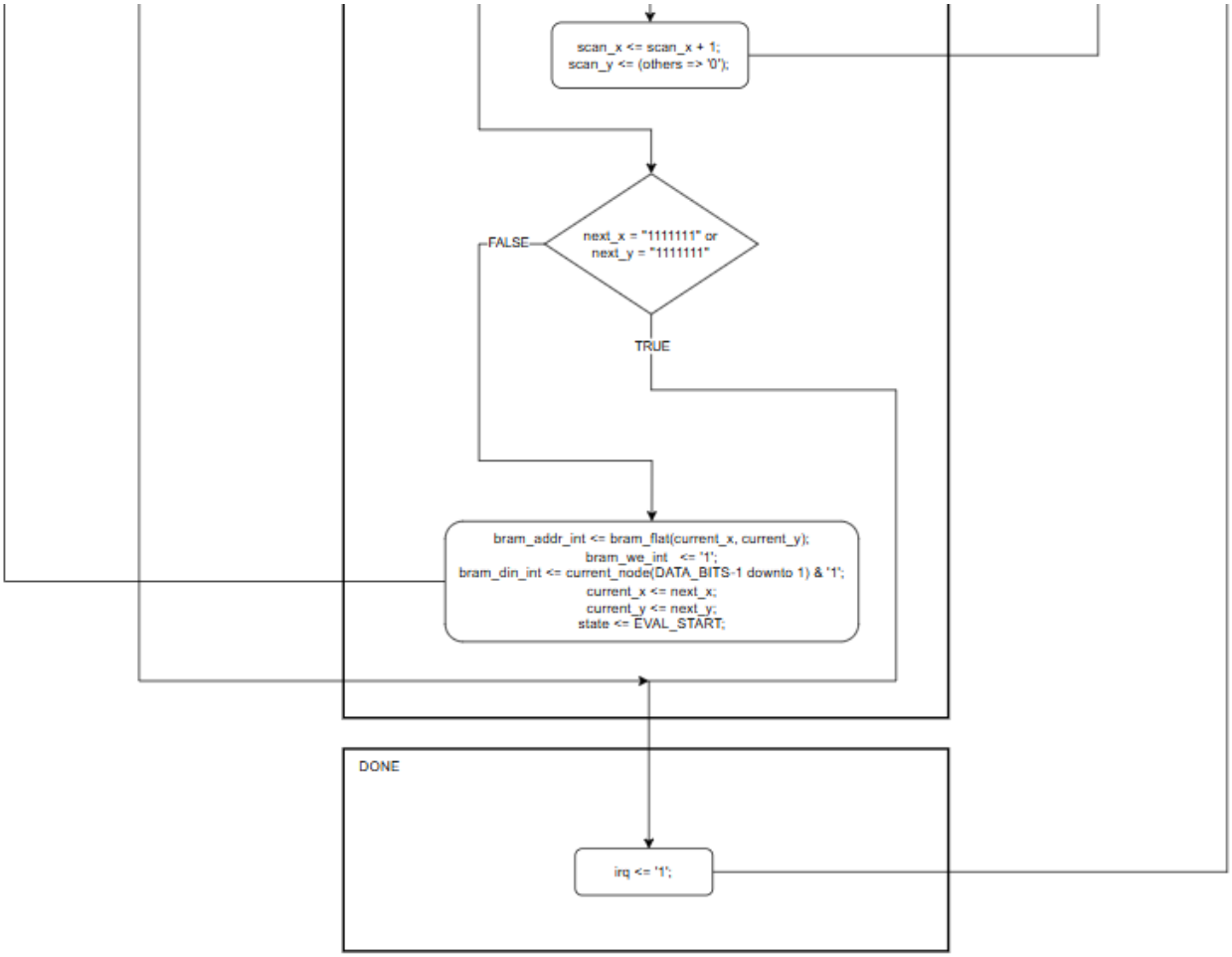

EVAL_START
eval_i <= current_x - 1;
eval_j <= current_y - 1;
EVAL_READ
eval_i = 127 or eval_i >= GRID_SIZE
or (eval_i = current_x and eval_j = current_y)
or eval_j >= GRID_SIZE
or eval_j = 127
TRUE
FALSE
bram_addr_int <= bram_flat(eval_i, eval_j);
bram_we_int <= '0';
EVAL_WAIT
state <= EVAL_PROCESS;
EVAL_PROCESS
v_dist := unsigned(bram_dout(23 downto 15));
v_visited := bram_dout(0);


v_dist = 300
FALSE
TRUE
v_new_data(23 downto 15) := std_logic_vector(to_unsigned(300, 9);
v_new_data(14 downto 8) := std_logic_vector(current_x);
v_new_data(7 downto 1) := std_logic_vector(current_y);
v_new_data(0) := v_visited;
bram_din_int <= v_new_data;
bram_addr_int <= bram_flat(eval_i, eval_j);
bram_we_int <= '1';
solved <= '1';
v_dist = 0
TRUE
v_new_data(23 downto 15) := std_logic_vector(heuristic(eval_i, eval_j, end_x, end_y));
v_new_data(14 downto 8) := std_logic_vector(current_x);
v_new_data(7 downto 1) := std_logic_vector(current_y);
v_new_data(0) := v_visited; --evaluate ne dira visited bit
bram_din_int <= v_new_data;
bram_addr_int <= bram_flat(eval_i, eval_j);
bram_we_int <= '1';
EVAL_WRITE
state <= EVAL_NEXT;


EVAL_NEXT
solved = '1'
TRUE
FALSE
eval_j < current_y + 1 or eval_j = 127
FALSE
TRUE
eval_j <= eval_j + 1;
eval_i < current_x + 1 or eval_i = 127
FALSE
TRUE
eval_i <= eval_i + 1;
eval_j <= current_y + 1;
scan_x <= (others => '0');
scan_y <= (others => '0');
min_dist <= (others => '1');
next_x <= (others => '1');
next_y <= (others => '1');


MOVE_READ
bram_addr_int <= bram_flat(scan_x, scan_y);
bram_we_int <= '0';
MOVE_WAIT
state <= MOVE_PROCESS;
MOVE_PROCESS
v_dist := unsigned(bram_dout(23 downto 15));
v_visited := bram_dout(0);
scan_x = current_x and
scan_y = current_y
FALSE
TRUE
current_node <= bram_dout;


v_visited = '0' and v_dist > 0 and
v_dist < 298 and v_dist < min_dist
FALSE
TRUE
min_dist <= v_dist;
next_x <= scan_x;
next_y <= scan_y;
MOVE_NEXT
scan_y < GRID_SIZE - 1
FALSE
TRUE
scan_y <= scan_y + 1;
scan_x < GRID_SIZE - 1
FALSE
TRUE



Rad akceleratora je upravljan konačnim automatom sa 12 stanja:

IDLE: Inicijalno stanje. Čeka upis na kontrolnu adresu (cpu_addr = "10" i cpu_we = '1'). Resetuje registre min_dist, next_x, next_y i solved fleg, i postavlja current_x i current_y na startne koordinate.

EVAL_START: Inicijalizuje eval_i i eval_j pomoćne registre, kako bi se započela evaluacija susednih čvorova.

EVAL_READ: Proverava da li su izračunati indeksi van granica mape (opseg 0–99) ili odgovaraju trenutnom čvoru. Ako jesu, prelazi se u EVAL_NEXT, u suprotnom se adresira BRAM.

EVAL_WAIT: Takt kašnjenja neophodan usled sinhronog čitanja iz BRAM memorije (read latency).

EVAL_PROCESS: Ukoliko je trenutno posmatrani čvor validan (neposećen i heuristika jednaka nuli, što znači da nije prethodno računata), računa heuristiku i prelazi u EVAL_WRITE. U ostalim slučajevima (END, WALL ili START čvor) preskače se upis i prelazi u EVAL_NEXT.

EVAL_WRITE: Troši jedan takt za upis novog stanja čvora u BRAM memoriju.

EVAL_NEXT: Inkrementira indekse susednih čvorova. Ako su obrađeni svi susedi trenutnog čvora, proverava fleg solved. Ako je putanja pronađena prelazi u DONE, a u suprotnom inicijalizuje promenljive za pretragu minimalnog čvora i prelazi u MOVE_READ.

MOVE_READ: Šalje adresu čvora (scan_x, scan_y) ka BRAM-u radi očitavanja čvora tokom pretrage mape za optimalnim sledećim čvorom.

MOVE_WAIT: Takt kašnjenja neophodan usled sinhronog čitanja iz BRAM memorije.

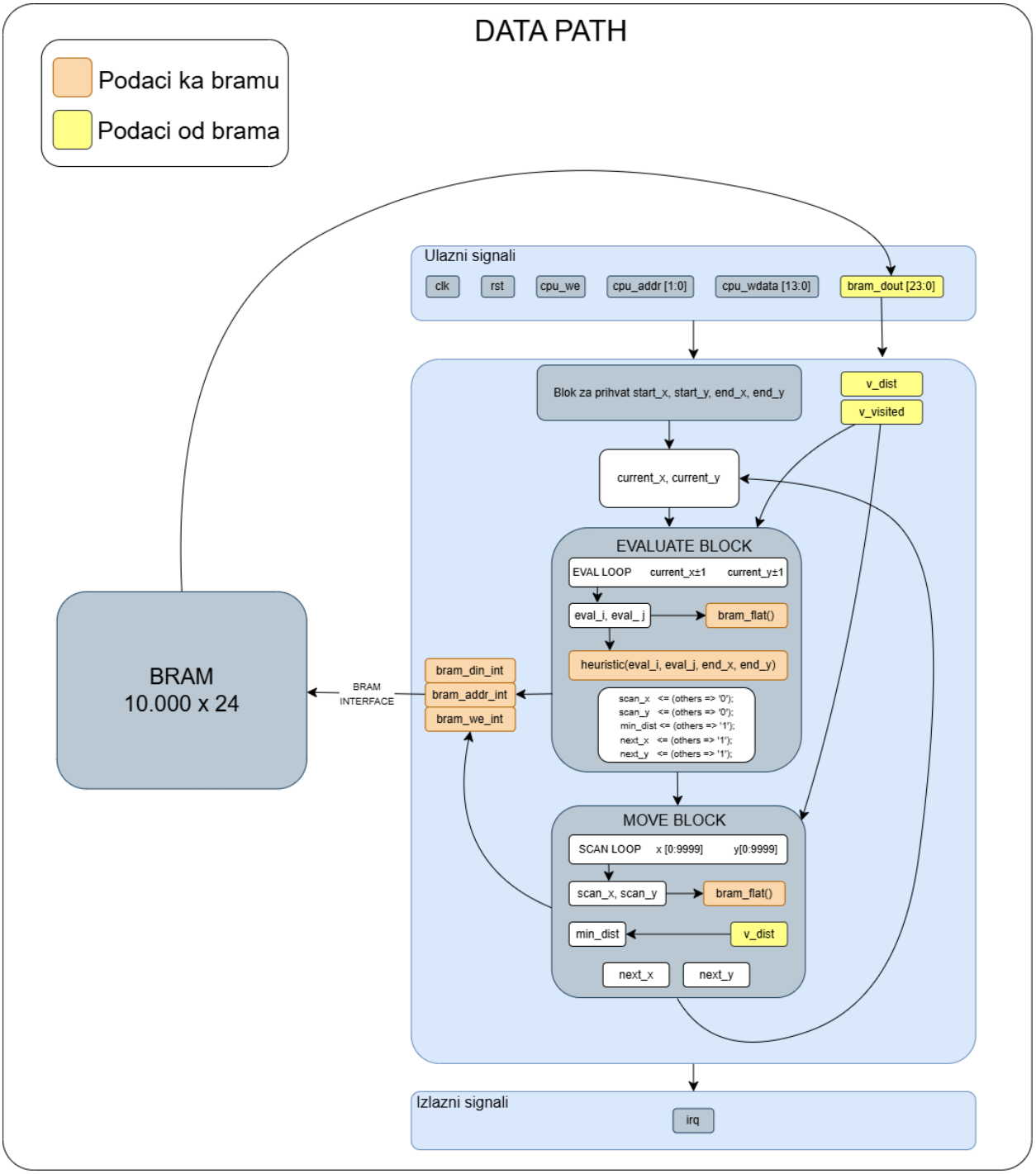
MOVE_PROCESS: Analizira pročitani čvor. Ako se pročitana adresa poklapa sa trenutnim čvorom, čuva se njegov sadržaj u registru current_node. Ako je čvor neposećen (visited = 0), validan (0 < distance < 298) i ima manju distancu od trenutno minimalne (distance < min_dist), ažuriraju se min_dist, next_x i next_y.

MOVE_NEXT: Prolazi kroz sve čvorove mreže (0 do 99 po x i y koordinatama). Po završetku skeniranja cele mape, ako next_x i next_y nisu promenjeni (ostali na vrednosti 127), mapa je nerešiva i prelazi u DONE. U suprotnom, upisuje visited = '1' za current_node, ažurira current_x <= next_x, current_y <= next_y i vraća se u stanje EVAL_START.

DONE: Generiše rastuću ivicu na prekidnom signalu irq <= '1' ka procesoru i vraća automatu stanje IDLE.

2.3 Datapath sistema

Na slici u nastavku prikazan je tok podataka kroz računske jedinice akceleratora, zajedno sa njenim ulazno-izlaznim signalima i interfejsom ka BRAM memoriji.

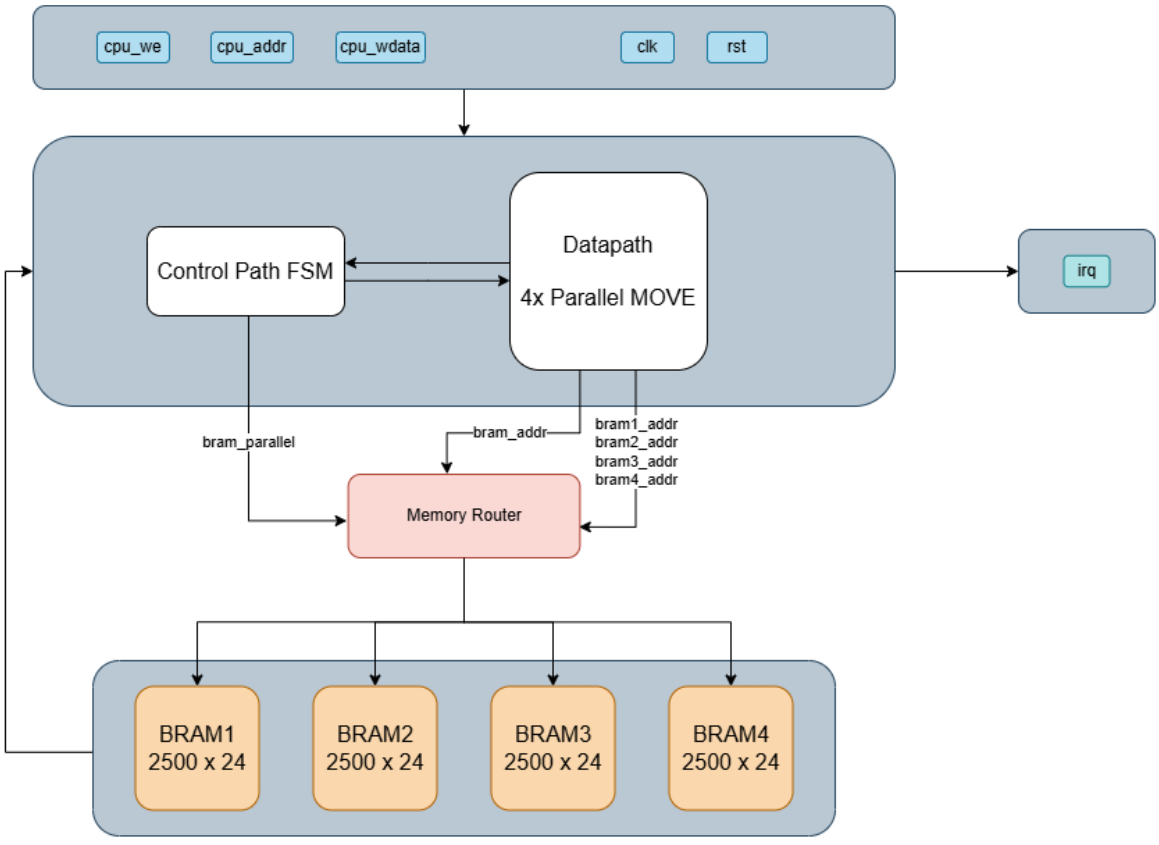


2.4 Paralelizacija pretrage mape

Većina vremena rada sistema troši se tokom pretrage mape za optimalnim sledećim čvorom, u MOVE fazi. Kako bi se rad akceleratora dodatno ubrzao, izvršena je četvorostruka paralelizacija ove pretrage. Pošto se pretraga vrši nad BRAM memorijom, koja ima samo jedan port ka IP bloku, bilo je potrebno podeliti memoriju u 4 dela. Podelu je bilo moguće izvršiti na prirodan način zbog odabranog načina organizacije podataka u memoriji opisanog na početku glave 2. Tako je BRAM memorija podeljena na 4 manje memorije koje redom sadrže redove mape (0 – 24), (25 – 49), (50 – 74), (75 – 99). Na taj način, omogućeno je istovremeno čitanje 4 čvora, kao i istovremeno pretraživanje sva četiri dela mape. Na kraju pretrage dobijaju se 4 lokalno optimalna čvora, koji se se međusobno porede kako bi se odredio optimalni sledeći čvor.

Posledica ove paralelizacije je bila potreba za uvođenjem dodatnog dekodiranja kada se u EVAL fazi pristupa memorji.

2.5 Blok dijagram sistema sa paralelizacijom pretrage



Kontrolni signal *bram_parallel* određuje da li se paralelno pristupa svim podblokovima memorije, ili se pristupa samo jednoj 14-bitnoj adresi, koja se dekodira u 12-bitnu adresu i šalje jednom od 4 bloka.

3. Interfejsi sistema

U nastavku je prikazana precizna tabela generičkih parametara i ulazno-izlaznih interfejsa VHDL modula pathfinder_ip.

3.1 Generički parametri sistema

Naziv	Tip	Podrazumevana vrednost	Opis
GRID_SIZE	integer	100	Dimenzija mape (100 x 100).
ADDR_BITS	integer	14	Broj bitova za adresiranje glavne BRAM memorije.
DATA_BITS	integer	24	Dužina reči memorije.

3.2 Ulazno-izlazni interfejsi

Naziv port-a	Smer	VHDL tip / širina	Aktivni nivo	Opis i funkcionalnost u kodu
clk	In	std_logic	Rastuća ivica	Glavni taktni signal.
rst	In	std_logic	‘0’	Sinhroni reset signal.
cpu_we	In	std_logic	‘1’	Dozvola upisa od strane procesora.
cpu_addr	In	std_logic_vector[1:0]	Nivo	Adresa registra upisa od strane procesora.
cpu_wdata	In	std_logic_vector[ADDR_BITS-1:0]	Nivo	Ulazni podaci od strane procesora.
bram_en	Out	std_logic	‘1’	Dozvola rada glavne BRAM-a.
bram_we	Out	std_logic	‘1’	Dozvola upisa u BRAM.
bram_addr	Out	std_logic_vector[ADDR_BITS-1:0]	Nivo	Adresni signal za BRAM.
bram_din	Out	std_logic_vector[DATA_BITS-1:0]	Nivo	Ulazni podaci BRAM-a.
bram_dout	In	std_logic_vector[DATA_BITS-1:0]	Nivo	Izlazni podaci BRAM-a.
irq	Out	std_logic	‘1’	Prekidni signal ka procesoru.
bram_parallel	Out	std_logic	‘1’	Kontrolni signal za paralelni pristup memoriji.
bram1_en .. bram4_en	Out	std_logic	‘1’	Dozvole rada BRAM-ova.
bram1_we .. bram4_we	Out	std_logic	‘1’	Dozvole upisa BRAM-ova.

bram1_addr .. bram4_addr	Out	std_logic_vector[11:0]	Nivo	Adresni signali BRAM-ova.
bram1_din .. bram4_din	Out	std_logic_vector[DATA_BITS-1:0]	Nivo	Podaci za upis u BRAM-ove.
bram1_dout .. bram4_dout	In	std_logic_vector[DATA_BITS-1:0]	Nivo	Izlazni podaci BRAM-ova.

4. Test benč i provera osnovne funkcionalnosti IP-a

Verifikacija osnovne funkcionalnosti akceleratora izvedena je pomoću VHDL test benča definisanog u fajlu *tb.vhd*. U njemu se instanciraju testirani IP blok, *pathfinder_ip.vhd*, i četiri BRAM memorije koje zajedno čine memoriju mape. Pošto u simulaciji ne postoji procesor, test benč ga oponaša, tako što upisuje potrebne vrednosti u registre i pokreće IP, čeka prekid i snima sadržaj memorije u tekstualni fajl, kako bi se koristio za kasnije analize.

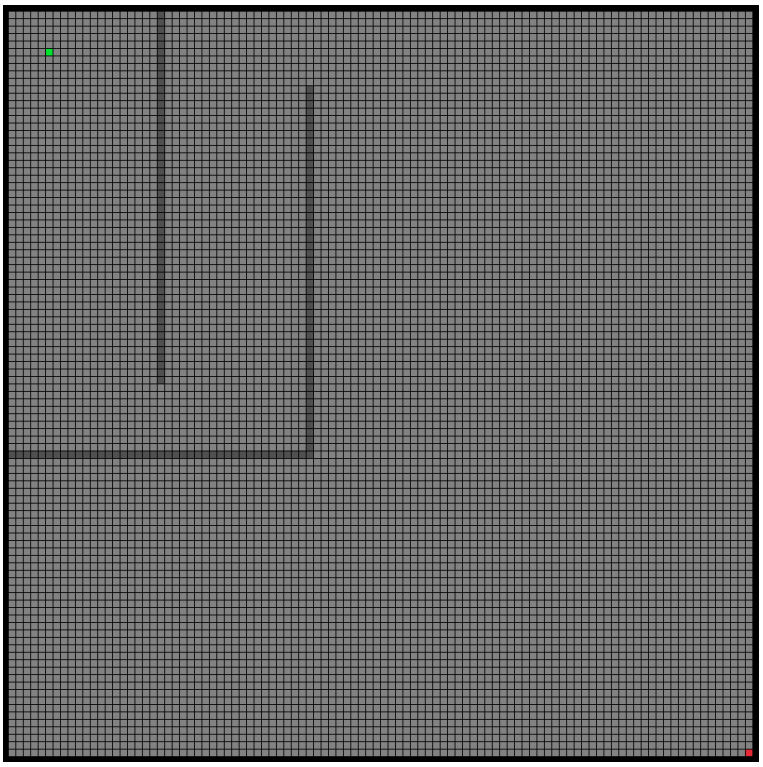
4.1 Tok simulacije

U nastavku se nalazi tok simulacije:

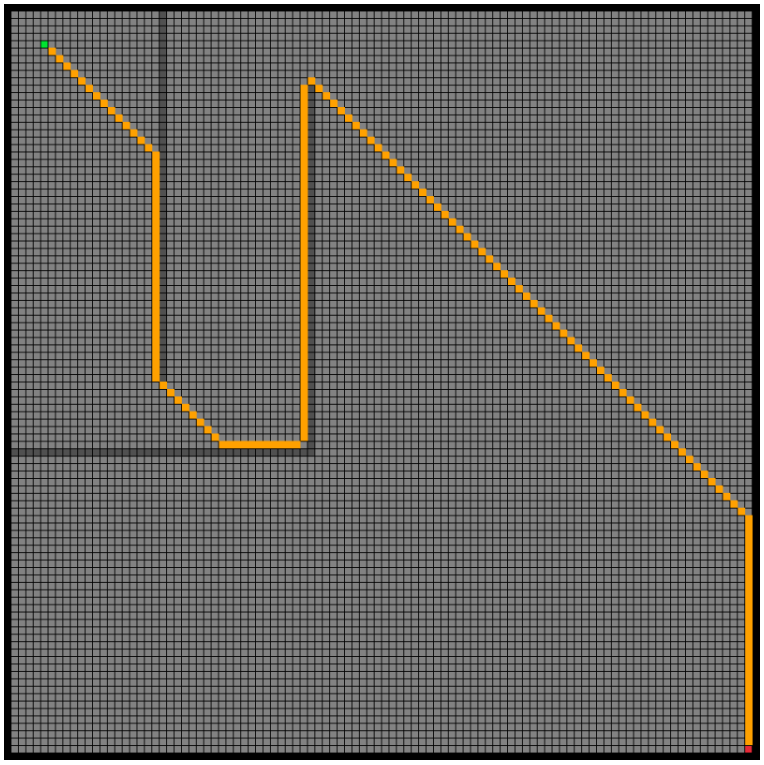
1. **Inicijalizacija –** signal rst_ip postavljen na '0' (reset aktivan), i svi upisni signali (wea1..wea4) su neaktivni.
2. **Učitavanje mape –** test benč otvara fajl definisan konstantom BRAM_INIT i čita iz njega liniju po liniju pomoću VHDL *std.textio* biblioteke. Svaka linija se konvertuje u 24-bitni *std_logic_vector* i upisuje na odgovarajuću adresu odgovarajuće banke (redosled adresa 0 do 9999 se linearno preslikava na banke 1–4, po 2500 adresa u svaku banku).
3. **Aktivacija IP-a –** nakon učitavanja, rst_ip prelazi u '1', čime IP blok izlazi iz reseta i FSM postaje aktivan (stanje IDLE).
4. **Upis START registra –** preko cpu_addr = "00" upisuje se koordinata početnog čvora.
5. **Upis END registra –** preko cpu_addr = "01" upisuje se koordinata ciljnog čvora.
6. **Start impuls –** sa cpu_addr = "10" i cpu_we = '1' prevodi FSM iz IDLE u EVAL_START i pokreće rešavanje mape.
7. **Čekanje na prekid –** test benč čeka irq_ip = '1', što odgovara trenutku kada FSM uđe u stanje DONE (mapa rešena ili utvrđeno da je nerešiva).
8. **Snimanje stanja memorije –** po prijemu prekida, test benč redom čita svih 10000 adresa i upisuje pročitane 24-bitne reči, liniju po liniju, u fajl definisan konstantom BRAM_DUMP.

4.2 Provera rezultata simulacije

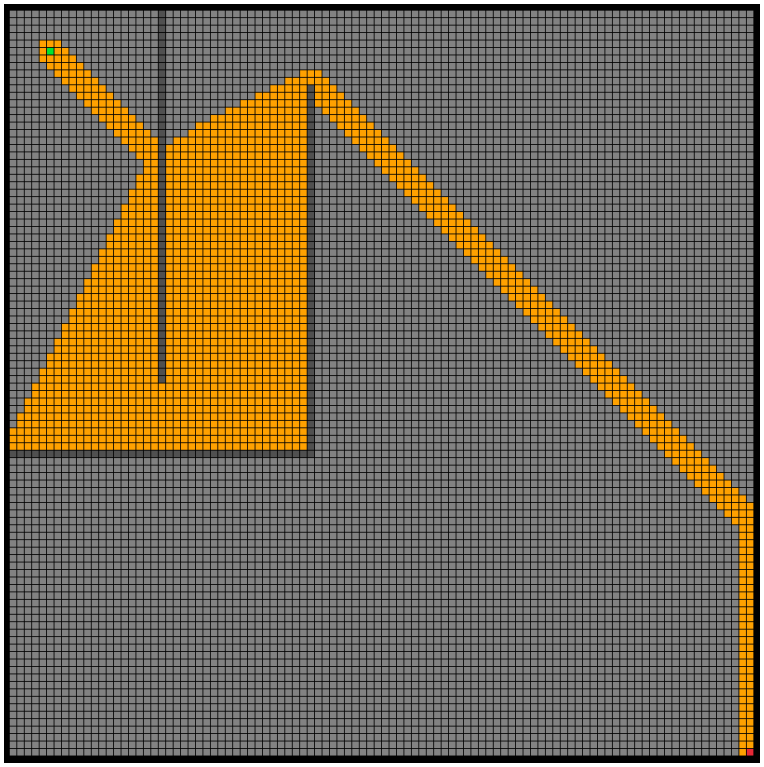
Kako je izlazni rezultat simulacije testnih slučajeva mapa u „sirovom“ obliku, bilo bi izuzetno vremenski zahtevno ručno proveriti sve nule i jednice iz izlaznog *bram_dump.txt* fajla kako bi se verifikovala ispravnost izlaza. Da bi se znatno olakšala provera rezultata test slučaja, implementiran je pomoćni softverski alat u *C++* koji iščitava tekstualni fajl koji sadrži podatke iz BRAM-a, i prikazuje ih grafički u istom stilu kao i u samoj softverskoj specifikaciji projekta. U poglavlju 1.2 već je naglašeno, da IP izvršava glavni, iterativni deo algortima, dok procesor kasnije u vrlo kratkom vremenu iz BRAM-a iščitava pronađenu putanju. Zbog toga se u grafičkom prikazu izlaza simulacije ne prikazuje narandžastom bojom konačna putanja, već svi pretraženi čvorovi. Pretraženi čvorovi su oni čija je heuristika izračunata tokom izvršavanja algoritma. Na osnovu vizuelnog prikaza pretraženih čvorova lako je zaključiti da li je IP došao do krajnjeg čvora ili nije. U nastavku prikazan je još jedan primer (pored primera iz 1.2) uspešno rešene mape:



Primer mape



Konačno rešenje mape - putanja



Izlaz IP-a – na osnovu stanja BRAM-a na kraju simulacije

5. Implementacija AXI interfejsa

Da bi akcelerator mogao da se poveže sa procesorskim sistemom, oko postojećeg IP bloka dodat je omotač sa AXI4-Lite slave interfejsom. Omotač je opisan u fajlu *pathfinder_axi.vhd*, dok se logika samog AXI protokola nalazi u fajlu *pathfinder_axi_slave_lite_v1_0_S00_AXI.vhd*. Unutrašnja logika akceleratora, opisana u prethodnim poglavljima, pri tome nije menjana. Ovakva podela omogućava da se *pathfinder_ip.vhd* instancira kao komponenta unutar omotača. Signali **cpu_we**, **cpu_addr** i **cpu_wdata** se više ne pobuđuju iz test benča, već ih generiše AXI registarski interfejs na osnovu upisa koje procesor obavlja preko magistrale.

5.1 Registarski interfejs

Procesor upravlja akceleratorom preko tri registra, koji su mapirani na donja dva bita AXI adrese:

Adresa	Naziv	Opis
"00"	START	Koordinate početnog čvora, u formatu X(6:0) & Y(6:0).
"01"	END	Koordinate ciljnog čvora, u istom formatu.
"10"	CONTROL	Upis na ovu adresu pokreće izvršavanje algoritma.

Koordinate se pakuju u 14 bita, tako što se sedam bita X koordinate spaja sa sedam bita Y koordinate. AXI4-Lite magistrala po specifikaciji koristi 32-bitnu reč, pa se gornjih 18 bita pri upisu koordinata ne koristi.

Upis na adresu CONTROL ne prenosi podatak, već služi kao impuls koji prevodi automat iz stanja IDLE u EVAL_START. Po završetku rada, akcelerator generiše prekid na signalu **irq_out**, koji je u omotaču izveden kao poseban izlazni port.

5.2 Generički parametri AXI omotača

Naziv	Tip	Podrazumevana vrednost	Opis
GRID_SIZE	integer	100	Dimenzija mape (100x100).
ADDR_BITS	integer	14	Broj bitova za adresiranje BRAM memorije.
DATA_BITS	integer	24	Dužina reči memorije
C_S00_AXI_DATA_WIDTH	integer	32	Širina AXI4-Lite magistrale podataka.
C_S00_AXI_ADDR_WIDTH	integer	4	Širina AXI4-Lite adresne magistrale.

5.3 Verifikacija AXI verzije

Za proveru AXI verzije napravljen je test benč *tb_axi_verify.vhd*. Za razliku od test benča iz poglavlja 4, koji je direktno pobuđivao registarske signale, ovaj test benč obavlja stvarne AXI4-Lite transakcije prema omotaču. Ostatak toka simulacije je isti: učitava se mapa,

upisuju se START i END koordinate, šalje se start impuls, čeka se prekid i snima se sadržaj memorije.

Izlazni fajl AXI verzije poređen je sa izlaznim fajlom paralelne verzije iz poglavlja 4, za oba testna slučaja. Sadržaji su bili identični, čime je potvrđeno da dodavanje AXI omotača ne menja ponašanje akceleratora. Ispravnost pronađene putanje dodatno je proverena praćenjem lanca roditeljskih čvorova od ciljnog do početnog čvora.

Napomena: Slika grafičkog prikaza izlaza AXI verzije identična je slikama iz poglavlja 4.2, pa se ovde ne ponavlja.

6. Pakovanje sistema u IP jezgra

Sistem je podeljen na četiri IP jezgra, koja se zatim povezuju u blok dizajnu. Pored glavnog akceleratora, napravljena su tri pomoćna modula koji rešavaju povezivanje akceleratora i procesora sa memorijom.

6.1 Glavno IP jezgro

Glavni akcelerator upakovan je pod nazivom **gbfs_hardware_accelerator**, dok naziv VHDL entiteta ostaje *pathfinder_axi*. Jezgro sadrži tri izvorna fajla: *pathfinder_axi.vhd*, *pathfinder_axi_slave_lite_v1_0_S00_AXI.vhd* i *pathfinder_ip.vhd*.

Prilikom pakovanja, alat je automatski prepoznao **s00_axi** kao AXI4-Lite interfejs, zajedno sa pratećim signalima takta i reseta. Signal **irq_out** je označen kao prekidni interfejs, čime je omogućeno njegovo direktno povezivanje na prekidni ulaz procesorskog sistema. Ostali portovi, koji vode ka memoriji, ostaju obični eksterni signali jer ne pripadaju nijednom standardnom interfejsu.

6.2 Pomoćna IP jezgra

bram_router obavlja usmeravanje sa strane akceleratora. Na ulazu prima ravan interfejs (**bram_addr**, **bram_din**, **bram_dout**) i paralelni interfejs (**bram1..4_***), a na izlazu daje četiri fizička interfejsa ka memorijskim bankama. Kontrolni signal **bram_parallel** bira koji od ta dva režima je aktivan, na način opisan u poglavlju 2.5.

bram_controller_router obavlja isti posao sa strane procesora. Prima jednu ravnu 14-bitnu adresu i dekodira je na četiri banke, tako što gornja dva bita biraju banku, a donjih dvanaest predstavlja adresu unutar nje. Modul je čisto kombinacioni, bez ijednog registra, pa ne unosi dodatno kašnjenje.

bram_controller_width_adapter prilagođava interfejs AXI BRAM kontrolera memoriji. Detalji ovog prilagođenja opisani su u poglavlju 7.3.

6.3 Verifikacija pomoćnih modula

Pošto su pomoćni moduli na kritičnoj putanji između procesora i memorije, svaki je proveren zasebnim test benčom pre nego što je uključen u blok dizajn.

Za **bram_controller_router** napravljen je test benč *tb_bram_controller_router.vhd*, koji meri koliko taktova prođe od postavljanja adrese do trenutka kada podatak postane validan. Uz

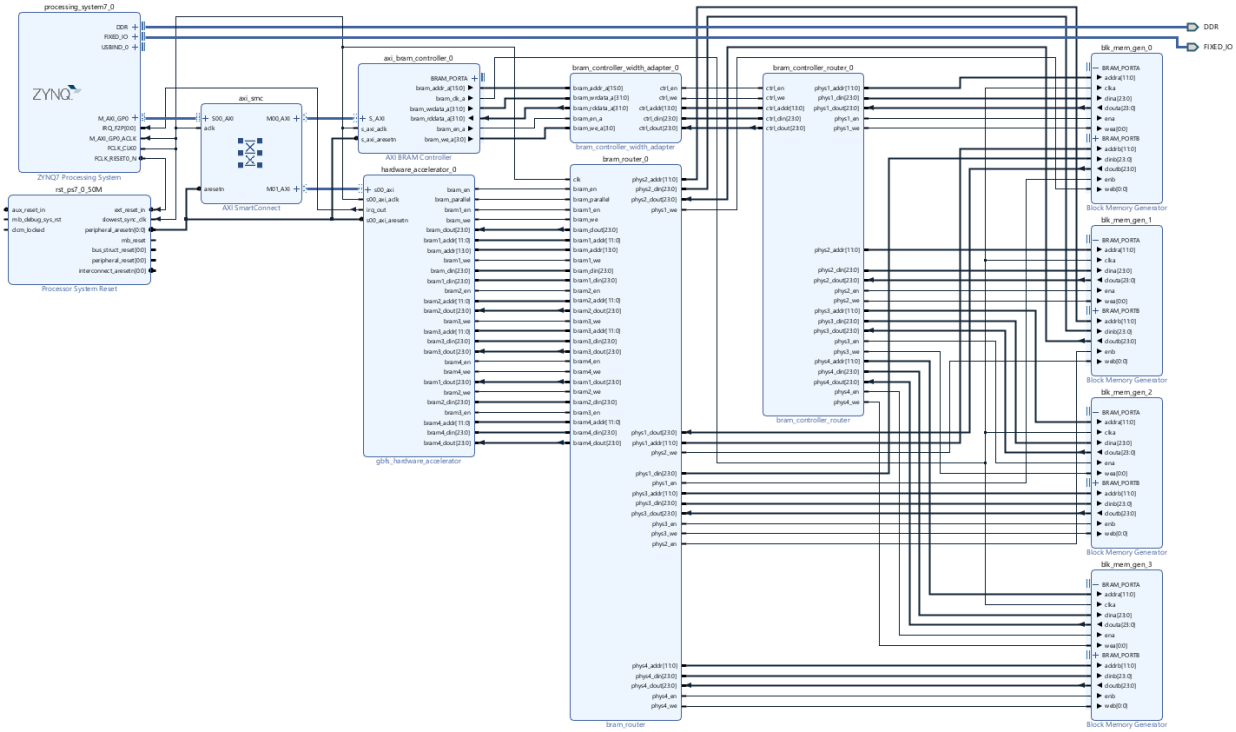
njega je napravljen i kontrolni test *tb_bram_direct_control.vhd*, koji istu meru sprovodi nad memorijom bez dekodera u putu. Poređenjem ta dva rezultata potvrđeno je da dekoder ne dodaje nijedan takt kašnjenja, odnosno da je latencija čitanja i dalje jedan takt, kao pri direktnom pristupu memoriji.

Ova provera je bila neophodna zato što automat akceleratora, u stanjima EVAL_WAIT i MOVE_WAIT, računa na tačno jedan takt kašnjenja pri čitanju. Svako odstupanje od te pretpostavke pokvarilo bi rad algoritma.

Za **bram_controller_width_adapter** napravljen je test benč *tb_bram_controller_width_adapter.vhd*, koji nezavisno proverava svih pet preslikavanja signala: odsecanje adrese, prolaz signala dozvole, spajanje bajt dozvola upisa u jedan bit, odsecanje podataka za upis i proširivanje pročitanih podataka nulama. Svih jedanaest provera je prošlo bez greške.

7. Blok dizajn sistema

Blok dizajn povezuje upakovana IP jezgra sa procesorskim sistemom i memorijom. Realizovan je u alatu Vivado 2024.1, za ploču Zybo Z7 sa čipom xc7z010clg400-1.



7.1 Topologija sistema

Procesorski sistem (ZYNQ7 Processing System) preko porta **M_AXI_GP0** vodi na AXI SmartConnect, koji saobraćaj deli na dva odredišta. Prvo odredište je AXI BRAM kontroler, preko koga procesor puni i čita mapu. Drugo odredište je **s00_axi** interfejs akceleratora, preko koga procesor upisuje koordinate i pokreće rad.

Memorija se sastoji od četiri Block Memory Generator bloka, po jedan za svaku banku iz poglavlja 2.4. Svaka banka je konfigurisana kao dvoportna memorija, čime se rešava pristup sa dve strane bez arbitraže. Port A svake banke vodi ka procesoru, kroz **bram_controller_width_adapter** i **bram_controller_router**. Port B vodi ka akceleratoru, kroz **bram_router**.

Ovakvom podelom, procesor i akcelerator pristupaju istoj memoriji nezavisno, svaki svojim portom. Pošto procesor puni mapu pre pokretanja algoritma, a rezultat čita tek po prijemu prekida, njihovi pristupi se u praksi ne preklapaju.

Ceo dizajn radi na jednom taktu, **FCLK_CLK0**, koji procesorski sistem generiše na 100 MHz.

7.2 Konfiguracija memorijskih blokova

Sva četiri memorijska bloka konfigurisana su na isti način:

Parametar	**Vrednost**
Mode	Stand Alone
Memory Type	True Dual Port RAM
Širina reči (Port A i Port B)	24 bita
Dubina (Port A i Port B)	2500
Common Clock	uključeno
Byte Write Enable	isključeno
Primitives Output Register	isključeno

Poslednja stavka je bitna za ispravan rad sistema. Uključen izlazni registar dodao bi drugi takt kašnjenja pri čitanju, a automat akceleratora, kao i oba dekodera, projektovani su i verifikovani pod pretpostavkom da je latencija tačno jedan takt.

Opcija Common Clock znači da Port A i Port B dele isti taktni ulaz, pa se povezuje samo **clka**, dok **clkb** ostaje namerno nepovezan.

7.3 Prilagođenje širine podataka

AXI BRAM kontroler i memorijske banke ne koriste iste širine signala, pa je između njih bilo potrebno umetnuti modul za prilagođenje. Razlika postoji na tri mesta:

Signal	**Kontroler**	**Memorija**	**Prilagođenje**
Adresa	16 bita	14 bita	Uzima se donjih 14 bita.
Podaci	32 bita	24 bita	Pri upisu se uzima donjih 24 bita, pri čitanju se gornjih 8 bita popunjava nulama.
Dozvola upisa	4 bita (po jedan po bajtu)	1 bit	Sva četiri bita se spajaju logičkim ILI.

Širina od 32 bita na strani kontrolera posledica je same AXI4-Lite specifikacije, koja tu vrednost fiksira, pa se nije mogla podesiti na 24. Konvencija po kojoj se korisni podaci nalaze u donjim bitima šire reči ista je kao kod registarskog interfejsa iz poglavlja 5.1.

Modul za prilagođenje je čisto kombinacioni, pa ni on ne utiče na latenciju čitanja.

7.4 Adresni prostor

Oba AXI odredišta mapirana su u adresni prostor procesora:

Blok	Bazna adresa	Opseg
AXI BRAM Controller	0x4000_0000	64 KB
GBFS Hardware Accelerator	0x4001_0000	4 KB

Opseg od 64 KB odgovara dubini memorije od 16384 reči, što pokriva svih 10000 lokacija mape sa rezervom. Akceleratoru je dovoljno 4 KB, pošto koristi samo tri registra.

7.5 Prekidni signal

Signal **irq_out** akceleratora povezan je na prekidni ulaz **IRQ_F2P** procesorskog sistema. Da bi taj ulaz uopšte postojao, u konfiguraciji procesorskog sistema uključena je opcija Fabric Interrupts. Ulaz je po specifikaciji 16-bitni, a akcelerator koristi njegov najniži bit.

Ovim je zaokružen predviđeni tok rada: procesor puni mapu i pokreće algoritam, akcelerator radi samostalno, a po završetku javlja procesoru prekidom umesto da procesor mora da ga ispituje u petlji.

7.6 Validacija dizajna

Provera blok dizajna (Validate Design) prošla je bez grešaka i kritičnih upozorenja. Prijavljena su dva upozorenja koja su razmotrena i prihvaćena.

Prvo upozorenje odnosi se na AXI SmartConnect. Alat je prepoznao da je sav saobraćaj kroz njega AXI4-Lite tipa, dakle male propusne moći, pa se blok automatski konfigurisao u režim Low-Area, odnosno u jednostavniju izvedbu koja troši manje resursa na čipu. Upozorenje se odnosi na burst transakcije, koje u tom režimu nisu podržane, ali ih AXI4-Lite po definiciji i ne koristi, pa se u ovom sistemu ne mogu pojaviti.

Drugo upozorenje javlja da interfejs **BRAM_PORTA** na AXI BRAM kontroleru nije povezan kao celina. Razlog je to što je taj interfejs namerno razložen na pojedinačne signale, kako bi prošao kroz modul za prilagođenje širine i dekoder. Posledica je da alati za inicijalizaciju memorije iz bitstream-a ne prepoznaju ovu memoriju, što u ovom sistemu nije potrebno, jer mapu upisuje procesor tokom rada.

8. Analiza integrisanog sistema

8.1 Analiza utrošenih resursa

Nakon sinteze i implementacije čitavog sistema na čipu xc7z010clg400-1, utrošeni su sledeći resursi:

Resurs	Iskorišćeno	Dostupno	Iskorišćenost
LUT	1588	17600	9,02%
LUTRAM	2	6000	0,03%
FF	1186	35200	3,37%
BRAM	10	60	16,67%

Iskorišćenost je niska po svim kategorijama, pa na ovom čipu ostaje dovoljno prostora za dalje proširenje, na primer za veći stepen paralelizacije pretrage.

Utrošak od 10 BRAM blokova posledica je organizacije memorije opisane u poglavlju 2.4. Četiri banke od po 2500 reči širine 24 bita ne poklapaju se tačno sa granicama fizičkih memorijskih blokova na čipu, pa je za njihovu realizaciju potrebno nešto više blokova nego što bi sledilo iz samog broja bita.

8.2 Analiza kritičnih putanja i maksimalne frekvencije

Sistem je implementiran sa ciljanom frekvencijom od 100 MHz, odnosno periodom od 10 ns. Rezultati vremenske analize:

Veličina	Vrednost
Worst Negative Slack (WNS)	0,098 ns
Total Negative Slack (TNS)	0,000 ns
Worst Hold Slack (WHS)	0,052 ns
Total Hold Slack (THS)	0,000 ns
Worst Pulse Width Slack (WPWS)	4,020 ns
Broj krajnjih tačaka koje ne zadovoljavaju	0
Ukupan broj krajnjih tačaka	3243

Svi vremenski zahtevi su ispunjeni, ali sa vrlo malom rezervom. Preostali slack od 0,098 ns znači da kritična putanja traje približno 9,9 ns, iz čega sledi da je maksimalna radna frekvencija sistema oko 101 MHz.

Pošto je rezerva tako mala, pri automatizovanoj izgradnji sistema koristi se strategija implementacije Performance_Retiming, opisana u poglavlju 9. Sa podrazumevanom strategijom, rezultat zna da bude na samoj granici, pa i neznatno negativan.

8.3 Određivanje propusne moći

Propusna moć sistema određena je merenjem u simulaciji. U test benč *tb_axi_verify.vhd* su dodata dva brojača. Prvi meri broj taktova između start impulsa i prekida, a drugi broji rastuće ivice signala **bram_parallel**, pošto svaka od njih odgovara jednom ulasku u paralelnu MOVE pretragu, dakle jednoj iteraciji algoritma.

Merenje je izvršeno na oba testna slučaja, pri radnoj frekvenciji od 100 MHz:

Testni slučaj	Taktova	Iteracija	Taktova po iteraciji	Vreme (ms)
bram_init.txt	18.175.718	1.811	10.036	181,76
bram_init_2.txt	69.993.736	6.974	10.036	699,94

Vreme rešavanja mape dobija se kao količnik broja taktova (N) i radne frekvencije (f):

$$T = \frac{N}{f}$$

Najvažniji rezultat merenja je to što je broj taktova po iteraciji identičan u oba slučaja, iako se ukupno vreme razlikuje skoro četiri puta. Razlog je što MOVE faza uvek pretražuje celu mapu, bez obzira na njen sadržaj. Uz četvorostruku paralelizaciju, to je 2500 čvorova po banci, uz četiri takta po čvoru, što daje ukupno 10.000 taktova, dok preostalih tridesetak taktova odlazi na EVAL fazu i prelaze između stanja.

Odavde sledi da cena jedne iteracije ne zavisi od mape, pa ukupno vreme zavisi isključivo od broja iteracija, odnosno od toga koliko čvorova algoritam proširi pre nego što stigne do cilja.

Zbog toga je za ovaj sistem najprirodnije propusnu moć izraziti brojem obrađenih čvorova u sekundi, jer je ta veličina nezavisna od mape:

$$P = \frac{f}{10.036} = 9.964 \text{ čvorova u sekundi}$$

8.4 Poređenje sa predviđanjima sa predmeta PEUNSN

Na predmetu „Projektovanje elektronskih uređaja na sistemskom nivou" procena performansi dobijena je alatom Vitis HLS. Sintetisani blok se sastojao od funkcija *moveNode()* i *evaluateSurroundingNodes()*, a alat je za njega prijavio kašnjenje od 10.007 ciklusa.

Pošto jedan poziv tog bloka odgovara jednoj iteraciji algoritma, ta vrednost se poredi sa brojem taktova po iteraciji izmerenim u poglavlju 8.3. Obe implementacije rade na 100 MHz, odnosno sa periodom od 10 ns:

Izvor	Taktova po iteraciji	Vreme po iteraciji
PEUNSN (procena)	10.007	10.007 x 10 ns = 100,07 us
PSDS (izmereno)	10.036	10.036 x 10ns = 100,36 us

Odstupanje iznosi:

$$\frac{10.036 - 10.007}{10.007} = 0{,}29\ \%$$

Time je uslov iz specifikacije projekta, po kome odstupanje ne sme biti veće od 20 %, ispunjen sa velikom rezervom.

Poklapanje je ovako dobro zato što obe implementacije rade isti posao na isti način. U oba slučaja jedna iteracija zahteva jedan pun prolaz kroz svih 10.000 čvorova mape. Alat Vitis HLS je taj prolaz protočno obradio, po jedan čvor u taktu, dok hardverska implementacija istu stvar postiže četvorostrukom paralelizacijom, obrađujući 2500 čvorova po banci uz četiri takta po čvoru.

8.4.1 Poređenje na nivou cele mape

Za merenje je korišćena ista mapa nad kojom je na predmetu PEUNSN izvršeno profajliranje, pa se poređenje može izvesti i za rešavanje cele mape. Profajliranje je pokazalo 1.742 poziva funkcije *moveNode()*, dok je hardverska implementacija napravila 1.811 iteracija.

Izvor	Iteracija	Ukupno taktova	Ukupno vreme
PEUNSN (procena)	1.742	1.742 x 10.007 = 17.432.194	174,32 ms
PSDS (izmereno)	1.811	18.175.718	181,76 ms

Odstupanje ukupnog vremena iznosi 4,27 %, što je takođe unutar dozvoljene granice.

Razlika u broju iteracija, 1.811 naspram 1.742, posledica je različitog izbora čvora u situacijama kada više neposećenih čvorova ima istu minimalnu heuristiku. Softverska verzija pretražuje mapu redom, a hardverska u četiri paralelna toka čiji se rezultati na kraju porede, pa izbor može ispasti drugačiji. Pošto GBFS ne garantuje jedinstvenu putanju, obe varijante daju ispravan rezultat.

9. Automatizacija izgradnje sistema

Ceo postupak izgradnje sistema automatizovan je jednom TCL skriptom, tako da se od izvornih VHDL fajlova, bez ijednog ručnog koraka u grafičkom okruženju, dobija gotov *.xsa* fajl spreman za Vitis.

Skripta obavlja sledeće korake redom:

1. **Pakovanje IP jezgara.** Za svaki od četiri modula pravi se privremeni projekat, dodaju se izvorni fajlovi, postavlja se top entitet i jezgro se pakuje u zajednički repozitorijum. Kod glavnog akceleratora dodatno se prepoznaju AXI4-Lite i prekidni interfejs.

2. **Kreiranje projekta i blok dizajna.** Pravi se nov projekat za ciljni čip, u njega se uključuje repozitorijum sa upakovanim jezgrima i kreira se prazan blok dizajn.
3. **Dodavanje i konfigurisanje blokova.** Instanciraju se procesorski sistem, akcelerator, AXI BRAM kontroler, oba dekodera, modul za prilagođenje širine, četiri memorijska bloka, SmartConnect i Processor System Reset, svaki sa parametrima iz poglavlja 7.
4. **Povezivanje.** Uspostavljaju se sve veze, uključujući taktnu i resetnu mrežu, i dodeljuje se adresni prostor iz poglavlja 7.4.
5. **Validacija i generisanje omotača.** Pokreće se provera dizajna i generiše se HDL omotač.
6. **Sinteza i implementacija.** Implementacija se pokreće sa strategijom Performance_Retiming, iz razloga navedenih u poglavlju 8.2.
7. **Generisanje bitstream-a i izvoz.** Na kraju se generiše bitstream i izvozi *.xsa* fajl sa uključenim bitstream-om.

Deo skripte koji se odnosi na blok dizajn nije pisan ručno, već je dobijen izvozom radnog dizajna komandom *write_bd_tcl*. Time je izbegnuta mogućnost greške u nazivima parametara pojedinih IP jezgara, kojih ima nekoliko stotina samo u konfiguraciji procesorskog sistema.

Skripta root projekta određuje na osnovu sopstvene lokacije, pa radi i kada se projekat premesti ili preuzme na drugi računar, bez ručnog podešavanja putanja.

10. Bare-metal aplikacija u Vitis alatu

Poslednji korak projekta predviđa programiranje ploče i proveru rada sistema kroz bare-metal aplikaciju. Ovaj deo nije doveden do kraja, pa se u nastavku navodi šta jeste urađeno i gde se stalo.

10.1 Urađeni deo

Bitstream je uspešno generisan, a hardverska platforma izvezena u *.xsa* fajl sa uključenim bitstream-om. U alatu Vitis napravljena je standalone platforma za jezgro ps7_cortexa9_0, koja je uspešno prevedena. Na njoj je napravljena i prevedena probna aplikacija, čime je potvrđeno da je lanac od Vivado projekta do Vitis okruženja ispravno postavljen.

10.2 Šta je preostalo

Preostalo je da se napiše sama aplikacija, koja bi trebalo da obavi sledeći tok: da upiše mapu u BRAM preko adresnog prostora AXI BRAM kontrolera, da upiše koordinate u START i END registre akceleratora, da pošalje start impuls na CONTROL adresu, da sačeka prekid i da na kraju pročita rezultat iz memorije i ispiše ga.

10.3 Razlozi zbog kojih deo nije završen

Prvi razlog je nemogućnost ponovnog korišćenja postojeće softverske specifikacije. Iz prvog dela projekta postoji gotova implementacija u jeziku C++, u terminalskoj i u grafičkoj

varijanti. Njena algoritamska logika se, međutim, ne može preneti, pošto upravo tu logiku sada izvršava akcelerator. Preuzeti se može samo format podataka, dok bi ostatak aplikacije morao da se piše iznova.

Drugi razlog je razlika u okruženju. Bare-metal aplikacija radi bez operativnog sistema, pa nema fajl sistem iz koga bi se mapa učitala na način na koji to radi terminalska verzija. Mapa bi morala da bude ugrađena u sam kod ili prenesena preko serijske veze. Iz istog razloga otpada i grafička varijanta specifikacije, jer standalone okruženje nema podršku za grafički prikaz, pa je jedini mogući izlaz konzolni ispis preko serijske veze.

Treći razlog je vreme. Rad na integraciji sistema, posebno na povezivanju memorije i verifikaciji da prilagođenje širine podataka i dekodiranje adresa ne narušavaju vremenske pretpostavke automata, oduzeo je više vremena nego što je planirano.

Treba naglasiti da funkcionalnost samog akceleratora nije ostala neproverena. Ispravnost rada potvrđena je simulacijom, i to u dve nezavisne verzije, sa i bez AXI omotača, čiji su izlazi bili identični, kao što je opisano u poglavljima 4.2 i 5.3. Ono što nedostaje je provera na fizičkoj ploči.